■ Python Concurrency — Lecture 2

Global Interpreter Lock (GIL)

Complete Study Notes | Interview Ready

#	Topic	Description
1	Background	What is CPython & why GIL was needed
2	Race Condition	The core problem GIL solves
3	Mutex & GIL	How the lock works step-by-step
4	GIL in Action	Threading code — slow CPU demo
5	Bypass with MP	Multiprocessing — fast CPU demo
6	GIL & I/O ops	When threading still shines
7	Pros & Cons	Trade-off analysis
8	Interview Q&A;	Top questions with model answers
9	Quick Revision	Cheatsheet for last-minute prep

■ Section 1 — Background: CPython & Why GIL Exists

What is CPython?

CPython is the original and most widely used implementation of Python, written in the C programming language. When people say 'Python', they almost always mean CPython.

Other implementations exist — **Jython** (Java), **PyPy** (Python), **IronPython** (.NET) — and notably, **GIL only exists in CPython**. This is an important interview point!

The Core Problem — Memory Management

CPython's memory management is NOT thread-safe by default.

Python tracks every object in memory using a **Reference Count** — a counter that records how many variables are pointing to an object. When the count drops to zero, the memory is freed.

Problem: If two threads simultaneously modify the reference count of the same object, the count can become corrupted — leading to memory leaks or premature deletion of objects still in use.

```
# How Reference Counting works:
x = [1, 2, 3]   # List created  → ref count = 1
y = x           # Another ref   → ref count = 2
del x           # One removed   → ref count = 1
del y           # Last removed  → ref count = 0 → Memory freed!

# THE PROBLEM WITH THREADS:
# Thread 1 reads ref count = 2, prepares to write 3
# Thread 2 reads ref count = 2, prepares to write 3
# Thread 1 writes 3
# Thread 2 writes 3  ← Should have been 4! Count is WRONG!
# Result: Memory corruption / crash
```

■ Section 2 — Race Condition

What is a Race Condition?

A **Race Condition** occurs when two or more threads access shared memory simultaneously and at least one is writing — producing unpredictable, incorrect results that depend on the exact timing of thread execution.

Almost all databases have mechanisms to handle race conditions. Python uses GIL as its primary defence.

Step-by-Step Race Condition Example

```
# Shared memory value = 4
# Thread 1 wants to change it to 5
# Thread 2 wants to change it to 3

# WITHOUT any lock — what can go wrong:
Step 1: Thread 1 reads  value → gets 4
Step 2: Thread 2 reads  value → gets 4  (Thread 1 hasn't written yet!)
Step 3: Thread 1 writes value → 5
Step 4: Thread 2 writes value → 3        (Overwrites Thread 1's result!)

Final value = 3
But maybe we expected 5  — WRONG RESULT!  Race condition occurred.

# This is non-deterministic — result changes every run!
```

Real-world counter race condition:

Thread 1: reads counter = 5, adds 1 → intends to write 6
Thread 2: reads counter = 5 (stale!), adds 1 → intends to write 6
Thread 1 writes 6 Thread 2 writes 6

Expected: 7 Actual: 6 One increment was silently lost!

■ Section 3 — Mutex and GIL Explained

What is a Mutex?

Mutex = Mutually Exclusive Lock

A mutex is a synchronisation primitive that ensures only **one thread** can access a protected resource at any given time.

Think of it like an ATM booth — only one person can be inside at a time. Everyone else waits in a queue outside. When the first person exits, the next one enters.

```
Mutex Lifecycle:
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Thread 1 arrives → Acquires mutex lock ■
Thread 1 works   → Reads / modifies shared data
Thread 2 arrives → Tries to acquire lock → BLOCKED ■
Thread 1 done    → Releases mutex lock ■
Thread 2 unblocked → Acquires mutex lock ■
Thread 2 works   → Now safely reads/modifies data
Thread 2 done    → Releases mutex lock ■
```

How GIL Works

GIL = Global Interpreter Lock

- **Global** — applies to the entire Python interpreter
- **Interpreter** — the engine that executes Python bytecode
- **Lock** — only the thread holding it can execute Python code

The GIL is a single mutex that must be held by any thread before it can execute Python bytecode. This prevents two threads from ever simultaneously modifying Python objects in memory.

```
GIL in action (CPU-bound scenario):
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Thread 1 → Acquires GIL ■ → executes Python code
Thread 2 → Wants GIL   → WAITING ■
Thread 1 → Periodically checks: should I release the GIL?
           (every ~100 bytecodes / 5ms in Python 3)
Thread 1 → Releases GIL ■
Thread 2 → Acquires GIL ■ → now executes Python code
... and so on — threads take turns, never truly parallel
```

Real-life analogy: Imagine a busy chai counter.
No matter how many baristas (threads) are working, **only one order can be processed at the counter at a time.**
The other baristas must wait for the counter to be free before they can process their customer's order.

■ Section 4 — GIL In Action: Threading Demo

File: 03_gil_threading.py

```
import threading
import time

def brew_chai():
    # Get and print the current thread's name
    thread_name = threading.current_thread().name
    print(f'{thread_name} started brewing...')

    # ■■ CPU-BOUND WORK ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    count = 0
    for _ in range(10_000_000):   # 10 million iterations
        count += 1                # Modifying shared-style counter
    # ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

    print(f'{thread_name} finished brewing!')

# Create two threads, give them names
thread1 = threading.Thread(target=brew_chai, name='Barista 1')
thread2 = threading.Thread(target=brew_chai, name='Barista 2')

start = time.time()

thread1.start()   # Begin execution
thread2.start()

thread1.join()    # Wait until thread1 finishes
thread2.join()    # Wait until thread2 finishes

print(f'Total time: {time.time() - start:.2f} seconds')
```

Expected Output

```
Barista 1 started brewing...
Barista 2 started brewing...
Barista 1 finished brewing!
Barista 2 finished brewing!
Total time: 5.20 seconds    ← SLOW!

# We expected ~2.5 seconds (2 cores doing half each)
# We got      ~5.2 seconds (effectively sequential!)
```

Why is it Slow? — The GIL Explanation

What we expected:
Thread 1 → Core 1 Thread 2 → Core 2 Both running in true parallel → ~2.5 seconds

What actually happened:
Both threads compete for the single GIL. Only one runs at any moment. The OS switches the GIL between them — but the total work is the same as running them sequentially → ~5.2 seconds.

Key insight: Adding more threads to a CPU-bound task in Python does NOT speed it up — it may actually slow it down due to GIL contention overhead.

■ Section 5 — GIL Bypass: Multiprocessing

File: 04_gil_multiprocessing.py

```
from multiprocessing import Process
import time

def crunch_number():
    print('Count process started...')
    count = 0
    for _ in range(10_000_000):   # Same 10 million iterations
        count += 1
    print('Count process ended!')

# ALWAYS required for multiprocessing — prevents infinite spawning!
if __name__ == '__main__':

    start = time.time()

    p1 = Process(target=crunch_number)   # Separate process — own GIL
    p2 = Process(target=crunch_number)   # Separate process — own GIL

    p1.start()   # Spawns a new OS process on its own CPU core
    p2.start()

    p1.join()    # Wait for p1 to complete
    p2.join()    # Wait for p2 to complete

    print(f'Total time: {time.time() - start:.2f} seconds')
```

Expected Output

```
Count process started...
Count process started...
Count process ended!
Count process ended!
Total time: 2.80 seconds    ← FAST! Almost 2x speedup

Threading:       ~5.20 seconds
Multiprocessing: ~2.80 seconds   — nearly 2x faster!
```

Why is Multiprocessing Faster?

```
Threading model:
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
ONE shared GIL
Thread 1 [GIL] ---> runs ---> releases GIL
Thread 2        waits  ---> [GIL] runs ---> releases
Effectively sequential. Total work = 10M + 10M iterations

Multiprocessing model:
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Process 1: OWN GIL -> Core 1 -> runs 10M iterations freely
Process 2: OWN GIL -> Core 2 -> runs 10M iterations freely
Both run TRULY IN PARALLEL on separate cores!
Total wall-clock time ≈ time for ONE process
```

Why if __name__ == '__main__' is Mandatory

The Problem without it:
When multiprocessing spawns a child process, the child imports the parent script. If process-creation code is at the top level (not guarded), the child also tries to spawn processes — leading to infinite recursion and this error:

`RuntimeError: An attempt has been made to start a new process before the current process has finished its bootstrapping phase.`

The Fix: Wrap all process-creation code inside `if __name__ == '__main__':` — this block only runs in the original script, not in spawned children.

Note: Threading does NOT need this guard — threads are lightweight units inside one process and do not re-import the script.

■ Section 6 — GIL & I/O Operations

The GIL is Released During I/O Waits!

This is the most important nuance about the GIL that many candidates miss.

When a thread performs an **I/O operation** (network request, file read, database query), it voluntarily **releases the GIL** while waiting for the response — because no Python objects are being modified during the wait.

This means other threads can run during that wait time, making **threading highly effective for I/O-bound tasks!**

```
# How threading helps with I/O operations:
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Thread 1 → Sends web request → WAITING for response
           GIL RELEASED during wait ↓
Thread 2 → Acquires GIL → Sends its web request → WAITING
           GIL RELEASED ↓
Thread 3 → Acquires GIL → Sends its web request → WAITING

All 3 requests are now in-flight simultaneously!
Responses arrive → each thread writes to its OWN memory
No shared memory conflict → No GIL bottleneck!

Sequential:   3 requests × 0.5s each = 1.5 seconds
With threads: ~0.5 seconds (all waiting in parallel)
```

Threading Use Case — Download Example

```
import threading, requests, time

def download(url):
    print(f'Starting: {url}')
    response = requests.get(url)    # I/O wait — GIL released here
    print(f'Done: {len(response.content)} bytes')

urls = [
    'https://httpbin.org/image/jpeg',
    'https://httpbin.org/image/png',
    'https://httpbin.org/image/svg',
]

start = time.time()
```

```
    threads = []
    for url in urls:
        t = threading.Thread(target=download, args=(url,))
        t.start()
        threads.append(t)
    for t in threads:
        t.join()
    print(f'All done in {time.time()-start:.2f}s')  # ~1.5s vs ~4.5s sequential
```

■ Section 7 — GIL Pros & Cons

Advantages of GIL

Advantage	Explanation
Memory Safety	Prevents race conditions on Python objects automatically
Simpler C Extensions	Third-party C libraries integrate safely without extra locks
Fast Single-threaded	No lock overhead for single-threaded programs
Reference Counting	Object cleanup (garbage collection) is simple and reliable
I/O Concurrency	Threading still works well for I/O-bound workloads

Disadvantages of GIL

Disadvantage	Explanation
No CPU Parallelism	Threads cannot utilise multiple cores for CPU-bound tasks
Bottleneck	All threads compete for one lock — contention slows them down
Misleading API	threading module exists but won't speed up CPU-heavy code
Workaround Required	Must use multiprocessing (more complex) for true parallelism
Memory Overhead (MP)	Each process has its own memory — more RAM needed

When to Use Threading vs Multiprocessing

Scenario	Best Choice	Python Tool
Database queries	Threading	threading / asyncio
REST API calls	Threading	threading / asyncio
File read / write	Threading	threading
Web scraping	Threading	threading
FastAPI web server	Async I/O	asyncio
Video / image processing	Multiprocessing	multiprocessing
Machine learning training	Multiprocessing	multiprocessing
Large mathematical computation	Multiprocessing	multiprocessing
Data pipeline (large CSV)	Multiprocessing	multiprocessing

■ Section 8 — Interview Questions & Model Answers

Q1. What is the GIL in Python?

The GIL, or Global Interpreter Lock, is a mutex in CPython that ensures only one thread can execute Python bytecode at any given time. It exists because CPython's memory management — specifically its reference counting garbage collector — is not thread-safe. Without the GIL, two threads could simultaneously corrupt an object's reference count, leading to memory errors. The GIL prevents this by serialising thread execution.

Q2. What is a Race Condition? How does GIL prevent it?

A race condition occurs when two threads read and write the same memory location concurrently, producing results that depend on unpredictable thread scheduling. For example, both threads might read counter=5, both add 1, and both write 6 — but the expected result was 7. The GIL prevents race conditions on Python objects by ensuring only one thread runs at a time, so no two threads can modify the same object simultaneously.

Q3. Does GIL mean Python threading is useless?

No. Threading is still highly useful for I/O-bound tasks. When a thread performs an I/O operation such as a network request or file read, it voluntarily releases the GIL while waiting. Other threads can then run during that wait time. For example, downloading three images with three threads takes roughly the same time as downloading one — all three requests are in-flight simultaneously. Threading only fails to provide a speedup for CPU-bound tasks.

Q4. How do you bypass the GIL for CPU-bound tasks?

Use the multiprocessing module instead of threading. Each spawned process is a completely separate OS process with its own Python interpreter and its own GIL. There is no sharing — so all processes run truly in parallel on separate CPU cores. The trade-off is that processes cannot share memory directly and require IPC mechanisms like Queue or Value to communicate.

Q5. Why must multiprocessing code use if __name__ == '__main__'?

When Python spawns a child process on Windows or macOS (spawn start method), the child imports the parent script to obtain the target function. Without the guard, the child re-executes the process-creation code, spawning more children — leading to infinite recursion and a RuntimeError. The guard ensures process-creation code only runs in the original script. Threading does not need this guard because threads live inside the same process and do not re-import the script.

Q6. Is GIL present in all Python implementations?

No. GIL is specific to CPython — the reference implementation written in C. Jython (Python on JVM) and IronPython (.NET) do not have a GIL and can achieve true thread-level parallelism. PyPy, another CPython alternative, historically had a GIL but the PyPy-STM project aims to remove it. In practice, CPython is used in almost all production environments, so GIL is a real concern for Python developers.

Q7. What is threading.current_thread().name and why use it?

threading.current_thread() returns the Thread object representing the currently executing thread. The .name attribute gives it a human-readable label. You can set a custom name when creating a thread: threading.Thread(target=func, name='Worker-1'). This is useful for debugging — log messages that include the thread name make it much easier to trace which thread is executing which code.

Q8. What is the difference between .start() and .join()?

.start() tells the thread to begin execution — it returns immediately and the thread runs concurrently. .join() blocks the calling thread until the target thread terminates. Without join(), the main program might exit before threads finish, producing incomplete results. You must always call start() before join() — calling join() on a thread that has not been started raises a RuntimeError.

■ Section 9 — Quick Revision Cheatsheet

Key Definitions — One Liners

```
GIL          = Global Interpreter Lock — one thread executes Python at a time
CPython      = Original Python (C-based) — the only major impl with GIL
Race Cond.   = Two threads corrupt shared memory due to unsynchronised access
Mutex        = Mutually Exclusive Lock — only one holder at a time
Reference Cnt= CPython counts how many vars point to each object
I/O Bound    = Task spends time WAITING  (network, disk, DB)
CPU Bound    = Task spends time COMPUTING (math, ML, video)
.start()     = Begin thread/process execution
.join()      = Block caller until thread/process terminates
threading.current_thread().name = Name of the running thread
```

GIL Behaviour Summary

Scenario	GIL Behaviour	Result
2 threads, CPU-bound task	Serialised	SLOW — no speedup
2 threads, I/O-bound task	Released on wait	FAST — concurrent
2 processes, CPU-bound task	Each has own GIL	FAST — truly parallel
1 thread (single-threaded)	Never contested	Fast — no overhead
threading + requests/DB calls	GIL released	Efficient
threading + heavy computation	GIL held	Bottleneck

Common Mistakes to Avoid

```
MISTAKE 1: Using threading for CPU-heavy tasks expecting a speedup
FIX      : Use multiprocessing for CPU-bound work

MISTAKE 2: Forgetting if __name__ == '__main__' in multiprocessing
FIX      : Always wrap process creation in this guard

MISTAKE 3: args=('value')  — this is NOT a tuple, it's just parentheses
FIX      : args=('value',)  — trailing comma makes it a tuple

MISTAKE 4: Calling .join() before .start()
FIX      : Always start() first, then join()
```

```
MISTAKE 5: Assuming GIL is present in all Python implementations

FIX      : GIL is CPython-specific; Jython/IronPython do not have it
```

Golden Rule for Interviews:

"If your task is **WAITING** → Threading is efficient (GIL released during I/O)"

"If your task is **COMPUTING** → Use Multiprocessing (each process has its own GIL)"